



PRESIDENCY UNIVERSITY

Department of Computer Science & Engineering

COURSE NAME: Object Oriented Programming

COURSE CODE: CSE205

PROJECT TITLE: E-Commerce Management System Using Java and SQLite

Submitted By:

Name: Fahad Fardin Younus

Student ID: 253174038

Submitted To:

Mostofa Kamal Sagor

Course Instructor

Department of Computer Science

Presidency University

SUBMISSION DATE: 7th August 2026

DECLARATION

I hereby declare that this project entitled "E-Commerce Management System Using Java and SQLite" is my own original work completed as part of the requirements of the course Object Oriented Programming (CSE205) at Presidency University.

The project has been developed independently using Java, Java Swing, SQLite, and other supporting technologies. All external resources, documentation, and references used during the development process have been properly acknowledged in the reference section. No unauthorized source code has been copied from any online repository or submitted by another student.

I understand that plagiarism or any form of academic misconduct may result in disciplinary action according to the university's academic regulations.

Signed,

Fahad Fardin Younus

253174038

Table of Contents

Chapter 1: Introduction	1
Chapter 2: Literature Review / Background	4
Chapter 3: System Requirements	7
Chapter 4: System Design	11
Chapter 5: OOP Concepts Implementation	17
Chapter 6: Implementation & Key Code Highlights	23
Chapter 7 Testing & Validation	28
Chapter 8 Results & Discussions	31
Chapter 9 Conclusions & Future Work	34
References	36
Appendices	37

List of Figures

Fig4.1 Architecture	11
Fig4.2 UML Class Diagram	12
Fig4.3 Inheritance & Interface Diagram	15
Fig4.4 ER Diagram	16
Fig 5.11 User class field encapsulation	17
Fig 5.12 Product Stock Management	17
Fig 5.13 User Class	18
Fig 5.14 Customer class extending user	18
Fig 5.15 Admin class extending user	18
Fig 5.16 CashPyment Class	18
Fig 5.17 CreditCardPay Class	19
Fig 5.18 Mobilepayment Class	19
Fig 5.19 ProductService Class	19
Fig 5.191 Abstract User Class	20
Fig 5.192 Payable Class	20
Fig 5.193 ProductService	21
Fig 5.194 CartService	21
Fig 5.195 StorefrontPane	21
Fig 5.196 Invalid quantity exception	22
Fig 5.197 PreparedStatement	22
Fig 5.198 ImageUtil	22
fig 6.1 SHA-256	23
fig 6.2 filtering	24
fig 6.3 All UI Designs	26
fig 6.4 Database	26

List of Tables

Table 1 Key OOP Concepts	5
Table 2 Functional Requirements	7
Table 3 Class Descriptions	12
Table 4 Database	15
Table 5 Package Structure	16
Table 6 Development Environment	23
Table 7 Test Class	28
Table 8 Bugs	30
Table 9 Requirement Fulfill Metrix	31
Table 10 OOP Concepts Coverage	32
Table 11 Team Contribution	37
Table 12 Development Log	37

CHAPTER 1: INTRODUCTION

1.1 Background

The rapid growth of information technology has transformed the way businesses conduct commercial activities. Traditional retail systems increasingly rely on digital platforms to manage products, customers, orders, and payments efficiently. E-commerce systems enable customers to browse products, place orders, and complete purchases from anywhere while allowing administrators to manage business operations through centralized software.

Many small and medium-sized businesses still depend on manual record keeping or fragmented software solutions, leading to data inconsistency, inefficient inventory management, and difficulty tracking customer orders. A well-designed desktop-based management system can significantly improve operational efficiency by integrating these activities into a single platform.

This project presents an E-Commerce Management System Using Java and SQLite, developed as a desktop application using Java Swing. The system provides role-based access for administrators and customers. Customers can register, browse products, add items to a shopping cart, manage wish lists, place orders, and track their purchase history. Administrators can manage products, customers, orders, reports, and inventory through a dedicated administrative interface.

The application demonstrates practical implementation of Object-Oriented Programming concepts while using SQLite as a lightweight relational database for persistent data storage.

1.2 Problem Statement

Many small businesses lack affordable software capable of managing inventory, customer accounts, shopping carts, and orders within a single application. Manual management of these activities often results in inaccurate inventory records, duplicated customer information, delayed order processing, and inefficient business operations.

Existing enterprise e-commerce platforms are frequently expensive, cloud-dependent, and unnecessarily complex for small organizations or academic learning purposes. Furthermore, many educational projects demonstrate Object-Oriented Programming concepts only theoretically without integrating them into a complete software solution.

This project addresses these issues by developing a desktop-based E-Commerce Management System that combines practical business functionality with proper implementation of Object-Oriented Programming principles. The system provides secure authentication, persistent database storage, role-based authorization, inventory

management, order processing, reporting, and an intuitive graphical user interface within a single application.

1.3 Project Objective

The objectives of this project are:

1. To develop a desktop-based E-Commerce Management System using Java Swing and SQLite.
2. To implement secure user authentication with separate administrator and customer roles.
3. To provide complete product management including adding, updating, deleting, searching, categorizing, and displaying products with images.
4. To implement shopping cart, wish list, checkout, payment, and order management functionalities.
5. To demonstrate core Object-Oriented Programming concepts including encapsulation, inheritance, abstraction, polymorphism, collections, exception handling, and modular package organization.
6. To provide persistent storage using SQLite so that all business information remains available after the application restarts.

1.4 Scope

The developed system focuses on managing the core operations of a desktop-based e-commerce application.

Included Scope

- Customer registration and login
- Administrator login
- Secure password hashing
- Role-based authorization
- Product management (CRUD)
- Product categories
- Product image management
- Shopping cart management
- Wish list functionality
- Checkout process
- Payment processing
- Order placement
- Order history
- Customer management
- Sales reporting and analytics
- SQLite database persistence
- Java Swing graphical user interface

Excluded Scope

The following features are beyond the scope of this project:

- Online payment gateway integration
- Email or SMS notifications
- Multi-vendor marketplace support
- Mobile application
- Cloud database hosting
- Real-time shipment tracking
- Online product reviews and ratings
- Barcode or QR code inventory management

1.5 Report Structure

This report is organized into nine chapters. Chapter 1 introduces the project background, objectives, scope, and overall motivation. Chapter 2 reviews Object-Oriented Programming concepts and existing e-commerce systems. Chapter 3 presents the functional and non-functional system requirements together with stakeholder analysis and use cases. Chapter 4 discusses the overall system design, including architecture, UML class diagrams, package organization, and database design. Chapter 5 explains how Object-Oriented Programming concepts have been implemented within the project using practical code examples. Chapter 6 describes the development environment, implementation details, graphical user interface, and database persistence mechanisms. Chapter 7 presents the testing strategy, test cases, edge-case validation, and resolved defects. Chapter 8 evaluates the completed system against the identified requirements while discussing achieved results and current limitations. Finally, Chapter 9 concludes the project by summarizing the outcomes, reflecting on the learning experience, and proposing future enhancements.

CHAPTER 2: LITERATURE REVIEW / BACKGROUND

2.1 Overview of Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that organizes software into objects containing both data (attributes) and behavior (methods). Compared to procedural programming, OOP improves code organization, reusability, maintainability, and scalability, making it suitable for developing complex software systems.

Java is a fully object-oriented language that provides built-in support for encapsulation, inheritance, polymorphism, abstraction, interfaces, exception handling, and collections. These features make Java an excellent choice for desktop and enterprise application development.

The four fundamental OOP principles are:

Encapsulation: Protects object data by using private attributes with controlled access through getter and setter methods. In this project, classes such as **User**, **Product**, and **Order** use encapsulation to ensure data integrity.

Inheritance: Allows classes to reuse common properties and behaviors. The project uses inheritance to share common user information while supporting different user roles.

Polymorphism: Enables the same interface or method to behave differently depending on the object. It is applied in the payment module through different payment implementations and overloaded service methods.

Abstraction: Hides implementation details while exposing essential functionality using abstract classes and interfaces, making the system easier to maintain and extend.

Overall, OOP provides a modular and maintainable structure that supports the development of a reliable E-Commerce Management System.

2.2 Similar Existing Systems

Several existing e-commerce platforms were reviewed during the system analysis phase.

Amazon is one of the world's largest online marketplaces, offering product management, order processing, inventory management, and secure payment services. Its separation of customer and administrator functions influenced the design of this project.

Daraz Bangladesh provides product browsing, shopping carts, order tracking, and customer account management. Features such as wishlists, shopping carts, and product categorization in this project were inspired by similar workflows.

Shopify is an e-commerce management platform that focuses on inventory, customer management, and sales reporting. The administrator module of this project follows a similar management-oriented approach while remaining suitable for a desktop application.

2.3 Why Java?

Java was selected because it strongly supports Object-Oriented Programming while offering platform independence through the Java Virtual Machine (JVM). Java Swing provides a rich set of components for building desktop graphical user interfaces, and JDBC enables seamless communication with databases.

SQLite was chosen as the database because it is lightweight, serverless, and easy to integrate with Java applications while providing reliable relational data storage.

The combination of Java Swing and SQLite offers an efficient and practical solution for developing a desktop-based E-Commerce Management System.

2.4 Key OOP Concepts Relevant to the Project

The E-Commerce Management System applies Object-Oriented Programming principles to satisfy its functional and non-functional requirements. Each concept contributes to the system's design, maintainability, and scalability.

OOP Concept	Application in the System	Related System Requirement
Encapsulation	Classes such as User, Product, and Order store their data as private attributes with controlled access through getter and setter methods.	Protect user information, maintain data integrity, and support secure data management.
Inheritance	The Admin and Customer classes inherit common properties and behaviors from the User class.	Support role-based authentication while reducing code duplication.
Polymorphism	Different payment methods implement common payment behavior, and overloaded service methods provide flexible operations.	Allow multiple payment options and improve system flexibility.

Abstraction	Abstract classes and interfaces define common behavior while hiding implementation details from the user interface.	Simplify system design and make future feature expansion easier.
Exception Handling	Custom exceptions such as DuplicateUsernameException and InvalidQuantityException validate user operations and prevent invalid transactions.	Improve system reliability by handling invalid inputs gracefully.

Chapter 3: System Requirements

3.1 Stakeholder Analysis

The E-Commerce Management System has two primary stakeholders: Administrators and Customers, each with specific responsibilities.

Administrator

Administrators manage the overall system and have privileged access to:

- Add, edit, and delete products
- Upload product images
- Manage customer accounts
- Monitor and update orders
- View sales reports
- Maintain inventory

Administrative features are protected through secure role-based authentication.

Customer

Customers interact with the storefront to purchase products. They can:

- Register and log in
- Browse, search, and filter products
- View product details
- Manage shopping cart and wishlist
- Complete checkout
- View order history
- Update profile information

Customers cannot access administrator functions.

3.2 Functional Requirements

The functional requirements describe the operations that the system must perform.

ID	Requirement
FR-01	The system shall allow new customers to register using their personal information.

FR-02	The system shall authenticate users using a username and hashed password.
FR-03	The system shall provide separate access privileges for administrators and customers.
FR-04	Customers shall browse, search, and filter products by category without logging in.
FR-05	Customers shall view detailed product information, including images, descriptions, prices, and stock availability.
FR-06	Customers shall add, update, and remove products from the shopping cart.
FR-07	Customers shall add or remove products from their wishlist for future purchases.
FR-08	Customers shall complete the checkout process by selecting a payment method and confirming the order
FR-09	Customers shall view their previous orders and current order status.
FR-10	Customers shall update their personal profile information, including email, phone number, and address.
FR-11	Administrators shall perform Create, Read, Update, and Delete (CRUD) operations on products, including uploading product images and assigning categories.
FR-12	Administrators shall view and manage customer accounts.
FR-13	Administrators shall monitor customer orders and update their processing status.
FR-14	The system shall support product image management and display images on the storefront.
FR-15	The system shall generate administrative reports, including total sales, orders, customers, and revenue statistics.
FR-16	The system shall store all application data permanently using an SQLite database.
FR-17	The system shall securely store user passwords using the SHA-256 hashing algorithm.
FR-18	The system shall validate all user inputs before performing database operations.

3.3 Non-Functional Requirements

Performance: Common operations such as login, product search, and navigation should complete within an acceptable response time.

Security: Passwords are stored using SHA-256 hashing, and administrator features are protected through role-based authorization.

Usability: The interface follows a consistent and user-friendly design suitable for both administrators and customers.

Maintainability: The application uses a modular package structure separating models, services, utilities, exceptions, and user interface components

3.4 Use Case Descriptions

Use Case 1 – Customer Registration

Actor: Customer

Precondition: User is not registered.

Main Flow:

1. Open the registration page.
2. Enter required information.
3. Submit the form.
4. System validates the data.
5. Password is hashed.
6. Customer account is created.

Postcondition: Customer account is successfully stored.

Use Case 2 – Customer Purchase

Actor: Customer

Precondition: Customer is logged in.

Main Flow:

1. Browse products.
2. Add products to cart.
3. Proceed to checkout.
4. Select a payment method.
5. Confirm the order.
6. System updates inventory and stores the order.

Postcondition: Order is successfully placed.

Use Case 3 – Product Management

Actor: Administrator

Precondition: Administrator is logged in.

Main Flow:

1. Open Product Management.
2. Add, edit, or delete a product.
3. Save changes.
4. System validates and updates the database.

Postcondition: Product information is updated.

Use Case 4 – Order Management

Actor: Administrator

Precondition: Administrator is authenticated.

Main Flow:

1. View customer orders.
2. Update order status.
3. Save changes.

Postcondition: Order status is updated.

Use Case 5 – Report Generation

Actor: Administrator

Precondition: Administrator is logged in.

Main Flow:

1. Open Reports.
2. View sales statistics and customer information.

Postcondition: Business reports are displayed.

Chapter 4: System Design

4.1 System Architecture

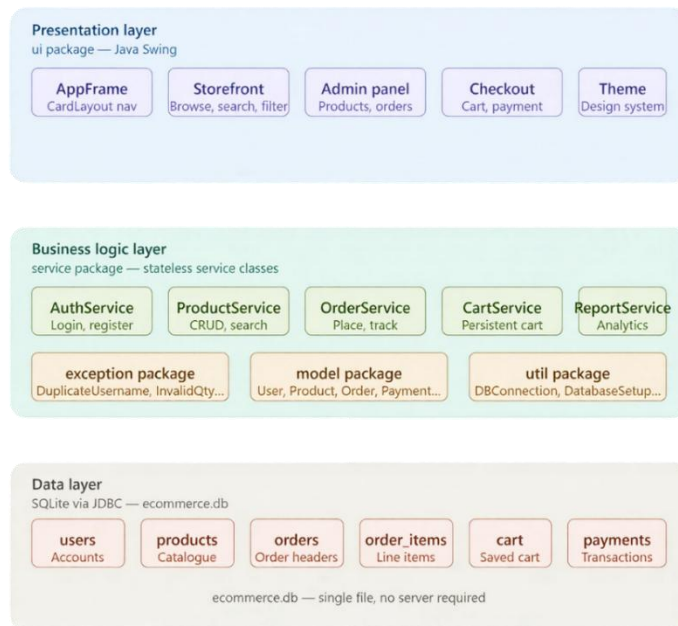


Fig4.1 Architecture

RegisterPanel, StorefrontPanel, AdminDashboardPanel, CustomerDashboardPanel), and frames (CartFrame, CheckoutFrame, OrdersFrame, WishlistFrame, ReportsFrame, ProductFormDialog, ProductDetailsDialog, ProfileFrame, CustomerManagementFrame, OrderDetailsFrame). A central Theme class provides all colours and fonts, and UiFactory provides reusable component creation methods.

Business Logic Layer (service package): Stateless service classes handle all business operations. AuthService manages authentication and password hashing. ProductService handles product CRUD with overloaded search methods. OrderService manages order placement with database transactions. CartService provides persistent cart management. WishlistService handles saved products. ReportService generates analytics. UserService manages customer profiles. PaymentService records payment results.

Data Layer (util + SQLite): DBConnection provides a singleton database connection. DatabaseSetup creates all tables on first launch. The SQLite database file (ecommerce.db) stores all data in six tables: users, products, orders, order_items, cart, payments, and wishlist

OmniCommerce follows a three-layer architecture that clearly separates concerns:

Presentation Layer (ui package): All Java Swing components including AppFrame (the main window using CardLayout for screen navigation), panels (LoginPanel,

4.2 UML Class Diagram

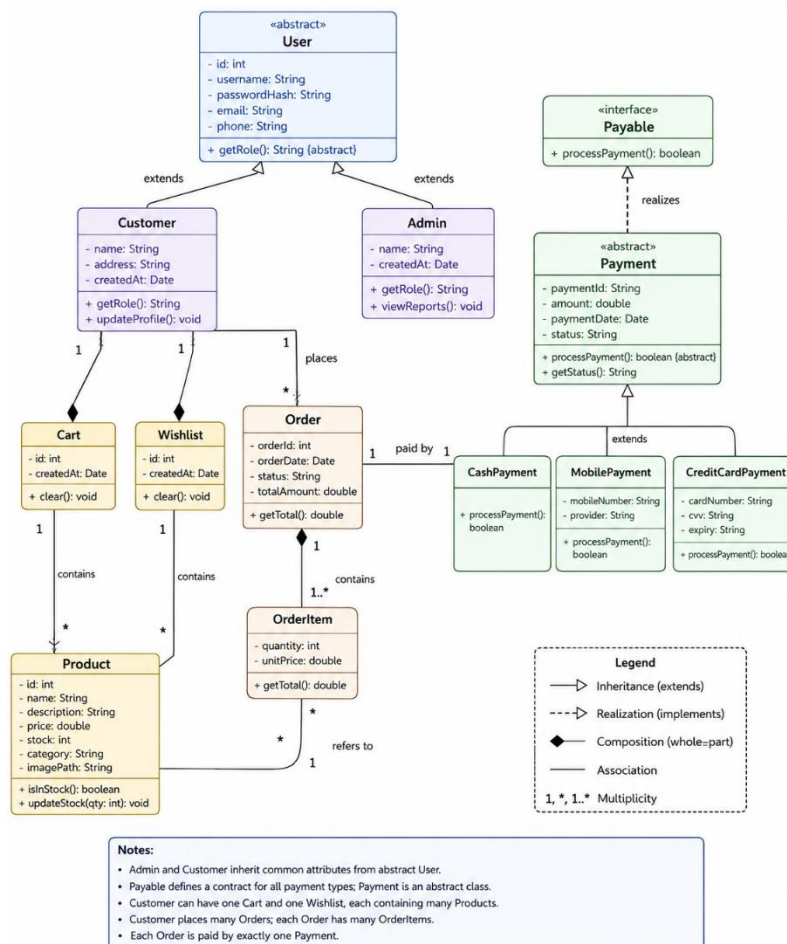


Fig.4.2 UML Class Diagram

4.3 Class Descriptions

The major classes used in the system are described below.

Class	Package	Purpose	Key Methods
User	model	Abstract base for all users	getRole() [abstract], setPassword(), setEmail()
Customer	model	Registered customer account	getRole() → "CUSTOMER"
Admin	model	Administrator account	getRole() → "ADMIN"
Product	model	Store product entity	reduceStock(), all getters/setters

The application consists of multiple interconnected classes representing users, products, orders, payments, and business services.

The UML Class Diagram illustrates:

- Class attributes
- Public methods
- Inheritance hierarchy
- Associations
- Dependencies
- Multiplicities

Order	model	Customer order record	getOrderId(), getStatus(), setStatus()
OrderItem	model	Single item within an order	getTotal() = quantity × price
Payment	model	Abstract payment base	processPayment() [abstract], getPaymentStatus()
CashPayment	model	Cash on delivery payment	processPayment() → always true
MobilePayment	model	bKash/Nagad payment	processPayment() with number validation
CreditCardPay	model	Credit card payment	processPayment() with card validation
Cart	model	In-memory cart helper	addItem(), calculateTotal(), clearCart()
AuthService	service	Login, registration, password hashing	login(), register(), addAdmin(), hashPassword()
ProductService	service	Product CRUD and search	addProduct(), getAllProducts(), searchProduct() [×2], updateProduct(), deleteProduct(), getProductById()
OrderService	service	Order placement and retrieval	placeOrder(), placeOrderAndReturnId(), getAllOrders(), getOrdersByCustomer(), updateOrderStatus()
CartService	service	Persistent cart management	addToCart(), getCart(), removeFromCart(), clearCart(), checkout(), getCartTotal()
WishlistService	service	Saved product management	add(), remove(), contains(), getAll()
ReportService	service	Analytics and reporting	getTotalSales(), getTotalOrders(), getTotalCustomers(), getLowStockProducts(), getSalesByMonth()

DBConnection	util	Singleton database connection	getConnection(), closeConnection()
DatabaseSetup	util	Table creation on first launch	createTables()
ImageUtil	util	Product image file management	copyProductImage(), scaledIcon()
Theme	ui	Central design system constants	Colour and font constants
UiFactory	ui.components	Reusable Swing component factory	primaryButton(), secondaryButton(), textField(), cardPanel()
AppFrame	ui	Main application window	showScreen(), completeLogin(), logout(), isAdmin()

4.4 Inheritance & interface Design

Figure 4.3 illustrates the inheritance and interface hierarchy implemented in the E-Commerce Management System. The User abstract class provides common attributes and operations that are inherited by the Customer and Admin classes, reducing code duplication and supporting role-based functionality.

The payment module combines abstraction and interfaces. The Payable interface defines a common payment contract, while the abstract Payment class stores shared payment information. Concrete classes such as CashPayment, MobilePayment, and CreditCardPay extend Payment and provide their own implementations of processPayment(). This design enables runtime polymorphism and allows new payment methods to be added with minimal modification to the existing code.

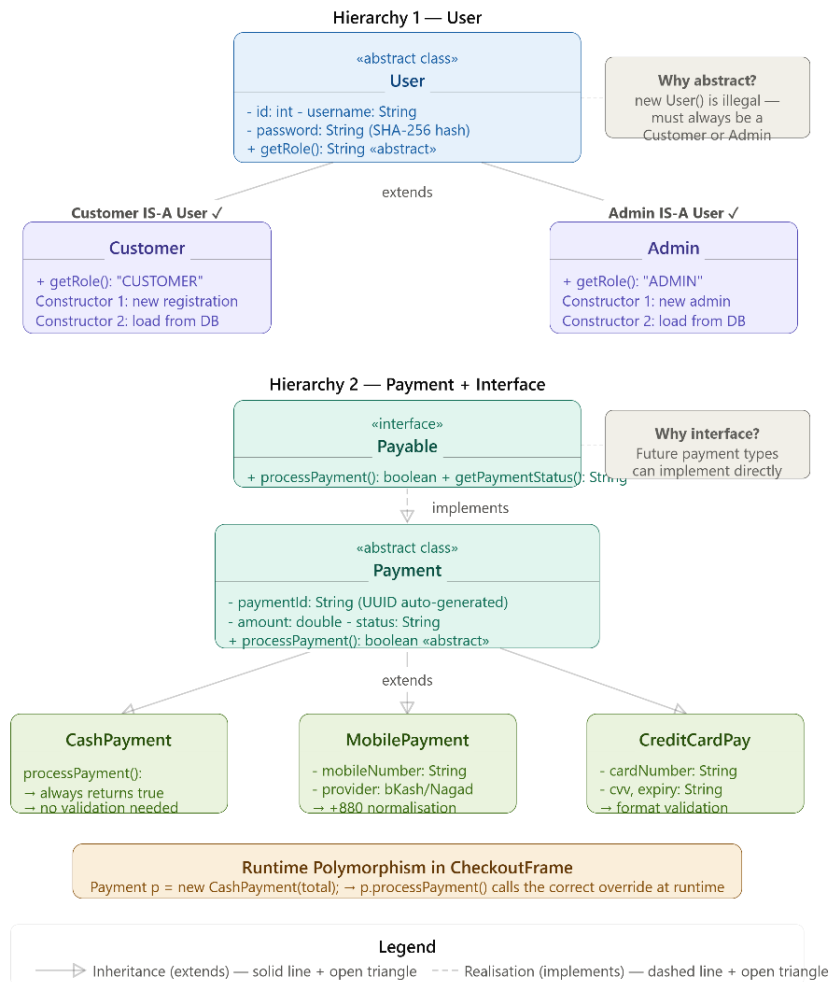


Fig4.3 Inheritance & Interface Diagram

4.5 Data Model

The SQLite database contains seven tables:

Table	Key Columns	Purpose
users	id, username (UNIQUE), password, role, email, phone, address	Stores all user accounts
products	id, name, description, price, stock, category, image_path	Stores all product catalogue entries
orders	id, customer_id (FK→users), total_amount, status, order_date	Stores order header records
order_items	id, order_id (FK→orders), product_id (FK→products), quantity, price	Stores individual line items per order
cart	id, customer_id (FK→users), product_id (FK→products), quantity	Stores persistent cart contents per customer

payments	id, order_id (FK→orders), amount, method, status, payment_date	Records payment outcome for each order
wishlist	id, customer_id (FK→users), product_id (FK→products) UNIQUE(customer_id, product_id)	Stores saved products per customer

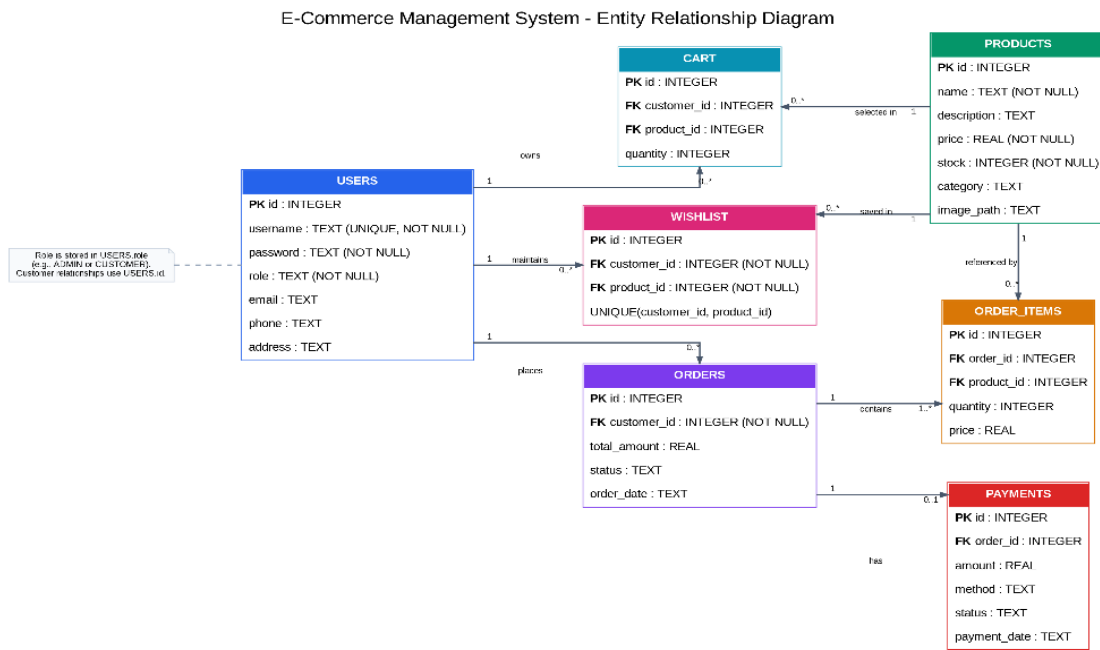


Fig4.4 ER Diagram

4.6 Package Structure

The project is organized into five packages following the guidelines' recommended structure:

Package	Contents	Responsibility
model	User, Customer, Admin, Product, Order, OrderItem, Payment, CashPayment, MobilePayment, CreditCardPay, Cart, Payable	Entity classes representing domain objects
service	AuthService, ProductService, OrderService, CartService, WishlistService, ReportService, UserService, PaymentService	Business logic and database operations
ui	AppFrame, Theme, panels/, frames/, components/	All graphical user interface components
exception	DuplicateUsernameException, InvalidQuantityException, PaymentFailedException, ProductNotFoundException	Custom domain specific exceptions
util	DBConnection, DatabaseSetup, ImageUtil	Shared utility and infrastructure classes

Chapter 5: OOP Concepts Implementation

5.1 Encapsulation

Encapsulation is applied throughout all model classes by declaring all fields private and providing controlled access through public getters and setters. This ensures that internal state cannot be modified arbitrarily from outside the class.

Example 1: User class field encapsulation:

```
public abstract class User {  
    private int id;  
    private String username;  
    private String password;  
    private String email;  
    private String phone;  
    private String address;  
    private String role;  
  
    public String getPassword(){  
        return password;  
    }  
  
    public void setPassword(String password){  
        this.password = password;  
    }  
    public void setEmail(String email){  
        this.email = email;  
    }  
    public void setPhone(String phone){  
        this.phone = phone;  
    }  
}
```

The password field is a key example of encapsulation in action. It is stored as a SHA 256 hash and can only be retrieved through getPassword() for comparison purposes. No other class can directly read or modify the raw field, preventing accidental exposure of sensitive data.

Fig 5.11 User class field encapsulation

Example 2 Product stock management:

```
public class Product {  
    private int id;  
    private String name;  
    private String description;  
    private double price;  
    private int stockQuantity;  
    private String category;  
    private String imagePath;  
  
    public void reduceStock(int quantity) {  
        if (quantity <= stockQuantity) {  
            this.stockQuantity -= quantity;  
        } else {  
            throw new IllegalArgumentException(s: "Not enough stock!");  
        }  
    }  
  
    public int getStockQuantity() {  
        return stockQuantity;  
    }  
}
```

Stock quantity cannot be set to any arbitrary value from outside the class. The reduceStock() method enforces a business rule: stock cannot go negative. This is encapsulation protecting data integrity.

Fig 5.12 Product Stock Management

5.2 Inheritance

Inheritance allows one class to reuse the properties and behavior of another class. It reduces code duplication while improving maintainability.

```

public abstract class User {
    private int id;
    private String username;
    private String password;
    private String email;
    private String phone;
    private String address;
    private String role;

    public User(int id, String username, String password, String email, String phone, String address, String role){
        this.id = id;
        this.username = username;
        this.password = password;
        this.email = email;
        this.phone = phone;
        this.address = address;
        this.role = role;
    }

    public abstract String getRole();
}

```

The **User** class stores information common to all users, including: Username, Password Hash, Email, Phone Number, Address, Role. The **Admin** and **Customer** classes inherit these common properties while implementing role specific responsibilities.

Fig 5.13 User Class

```

public class Customer extends User {

    public Customer(String username, String password, String email, String phone, String address) {
        super(id: 0, username, password, email, phone, address, role: "CUSTOMER");
    }

    public Customer(int id, String username, String password, String email, String phone, String address) {
        super(id, username, password, email, phone, address, role: "CUSTOMER");
    }

    @Override
    public String getRole() {
        return "CUSTOMER";
    }
}

```

The **super()** call in constructors demonstrates the correct use of the **super** keyword to initialize inherited fields.

Fig 5.14 Customer class extending user

```

public class Admin extends User {

    public Admin(String username, String password, String email, String phone, String address) {
        super(id: 0, username, password, email, phone, address, role: "ADMIN");
    }

    public Admin(int id, String username, String password, String email, String phone, String address) {
        super(id, username, password, email, phone, address, role: "ADMIN");
    }

    @Override
    public String getRole() {
        return "ADMIN";
    }
}

```

Fig 5.15 Admin class extending user

5.3 Polymorphism (Method Overriding)

Runtime polymorphism is demonstrated through the `processPayment()` method. All three payment types override this method with completely different behaviour:

```

public class CashPayment extends Payment {
    public CashPayment(double amount) {
        super(amount);
    }

    @Override
    public boolean processPayment() {
        System.out.println(x: "Cash payment processing...");
        setStatus(status: "PAID");
        return true;
    }
}

```

Fig 5.16 CashPymnt Class

```

public class CreditCardPay extends Payment{

@Override
public boolean processPayment() {
    // Validate card number - must be 16 digits
    if (cardNumber == null || cardNumber.length() != 16) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid card number!");
        return false;
    }

    if (cvv == null || cvv.length() != 3) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid CVV!");
        return false;
    }

    if (expiry == null || !expiry.matches(regex: "\\d{2}/\\d{2}")) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid expiry date! Use MM/YY format.");
        return false;
    }

    if (Math.random() < 0.9) {
        setStatus(status: "PAID");
        System.out.println(x: "Credit card payment successful!");
        return true;
    } else {
        setStatus(status: "FAILED");
        System.out.println(x: "Card declined!");
        return false;
    }
}
}

```

Fig 5.17 CreditCardPay Class

```

public class MobilePayment extends Payment {

@Override
public boolean processPayment() {
    if (mobileNumber == null) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid mobile number!");
        return false;
    }

    // Strip +880 if present and replace with 0
    String normalizedNumber = mobileNumber;
    if (mobileNumber.startsWith(prefix: "+880")) {
        normalizedNumber = "0" + mobileNumber.substring(beginIndex: 4);
    }

    // Now check it's 11 digits starting with 01
    if (normalizedNumber.length() != 11 || !normalizedNumber.startsWith(prefix: "01")) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid Bangladesh mobile number!");
        return false;
    }

    // Validate provider
    if (!provider.equals(anObject: "bKash") && !provider.equals(anObject: "Nagad")) {
        setStatus(status: "FAILED");
        System.out.println(x: "Invalid provider! Use bKash or Nagad.");
        return false;
    }
}
}

```

Fig 5.18 Mobilepayment Class

5.4 Polymorphism (Method overloading)

ProductService demonstrates method overloading with both addProduct() and searchProduct() methods having multiple signatures:

```

public class ProductService {
    public static void addProduct(String name, String description, double price, int stock, String category) {
        addProduct(name, description, price, stock, category, imagePath: null);
    }

    public static void addProduct(String name, String description, double price, int stock, String category, String imagePath) {...}

    public static List<Product> getAllProducts() {...}

    public static List<Product> searchProduct(String name) {
        return findProducts(sql: "SELECT * FROM products WHERE name LIKE ? ORDER BY id DESC", "%" + name + "%", second: null);
    }

    public static List<Product> searchProduct(String name, String category) {
        return findProducts(
            sql: "SELECT * FROM products WHERE name LIKE ? AND category LIKE ? ORDER BY id DESC", "%" + name + "%", "%" + category + "%");
    }

    private static List<Product> findProducts(String sql, String first, String second) {
        List<Product> products = new ArrayList<>();
        try {...}
    } catch (SQLException e) {...}
    return products;
}

public static void updateProduct(int id, String name, String description, double price, int stock, String category) {...}

public static void updateProduct(int id, String name, String description, double price, int stock, String category, String imagePath) {...}

```

Fig 5.19 ProductService Class

Method overloading improves readability because similar operations share the same method name while accepting different arguments.

5.5 Abstraction (Abstract Class)

The User abstract class prevents instantiation of incomplete objects and define contracts that subclasses must honour:

```
public abstract class User {
    private int id;
    private String username;
    private String password;
    private String email;
    private String phone;
    private String address;
    private String role;

    public User(int id, String username, String password, String email, String phone, String address, String role){
        this.id = id;
        this.username = username;
        this.password = password;
        this.email = email;
        this.phone = phone;
        this.address = address;
        this.role = role;
    }

    public abstract String getRole();
}
```

Fig 5.191 Abstract User Class

new User() is a compile error you must create a Customer or Admin. This prevents accidentally creating a "generic user" with no defined role, which would break the role based access control system.

5.6 Abstraction (Interface)

The Payable interface defines a contract that any payable entity must implement:

```
package model;

public interface Payable {
    boolean processPayment();
    String getPaymentStatus();
}
```

Fig 5.192 Payable Class

Payment implements Payable, and all three payment subclasses inherit this implementation. The interface separates what a payment object does (the contract) from how it does it (the implementation). In the future, a new payment type such as CryptoPayment could implement Payable directly without extending Payment, demonstrating the flexibility that interfaces provide over abstract classes alone.

5.7 Collections Framework

Java Collections are used extensively throughout OmniCommerce for managing lists of domain objects:

```

private static List<Product> findProducts(String sql, String first, String second) {
    List<Product> products = new ArrayList<>();
    try {
        Connection conn = DBConnection.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        if (first != null) stmt.setString(parameterIndex: 1, first);
        if (second != null) stmt.setString(parameterIndex: 2, second);
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) products.add(mapProduct(rs));
        stmt.close();
    } catch (SQLException e) { ...
    }
    return products;
}

```

Fig 5.193 ProductService

```

public static List<OrderItem> getCart(int customerId) {
    List<OrderItem> cartItems = new ArrayList<>();
    try {
        Connection conn = DBConnection.getConnection();
        String sql = "SELECT c.quantity, p.id, p.name, p.description, " + "p.price, p.stock, p.category, "
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setInt(parameterIndex: 1, customerId);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            int quantity = rs.getInt(columnLabel: "quantity");
            int id = rs.getInt(columnLabel: "id");
            String name = rs.getString(columnLabel: "name");
            String description = rs.getString(columnLabel: "description");
            double price = rs.getDouble(columnLabel: "price");
            int stock = rs.getInt(columnLabel: "stock");
            String category = rs.getString(columnLabel: "category");
            String imagePath = rs.getString(columnLabel: "image_path");

            Product product = new Product(id, name, description, price, stock, category, imagePath);
            cartItems.add(new OrderItem(product, quantity));
        }
        stmt.close();
    } catch (SQLException e) {
        System.out.println("Cart retrieval error: " + e.getMessage());
    }
    return cartItems;
}

```

Fig 5.194 CartService

```

private void refreshCategories() {
    Object selected = categoryBox.getSelectedItem();
    Set<String> categories = new LinkedHashSet<>();
    for (Product product : allProducts) {
        if (product.getCategory() != null && !product.getCategory().isBlank()) {
            categories.add(product.getCategory());
        }
    }
    categoryBox.removeAllItems();
    categoryBox.addItem(item: "All Categories");
    categories.stream().sorted(String.CASE_INSENSITIVE_ORDER).forEach(categoryBox::addItem);
    if (selected != null) categoryBox.setSelectedItem(selected);
    if (categoryBox.getSelectedIndex() < 0) categoryBox.setSelectedIndex(anIndex: 0);
}

```

Fig 5.195 StorefrontPanel

The StorefrontPanel uses a LinkedHashSet to collect product categories, ensuring each category appears only once while preserving insertion order for display in the filter dropdown. This demonstrates awareness of when different collection types are appropriate.

5.8 Exception Handling

OmniCommerce defines four custom exception classes in the exception package:

These exceptions carry contextual data beyond a simple message. The catch block in CheckoutFrame can call `e.getRequestQuantity()` and `e.getAvailableQuantity()` to display a specific, helpful error message to the user.

```

public class InvalidQuantityException extends Exception{
    private int requestedQuantity;
    private int availableQuantity;

    public int getRequestedQuantity(){...}
    public int getAvailableQuantity(){...}

    public InvalidQuantityException(int requestedQuantity, int availableQuantity){
        super("Invalid quantity! Requested: " + requestedQuantity + " but only " +
            this.availableQuantity + " available");
        this.availableQuantity = availableQuantity;
        this.requestedQuantity = requestedQuantity;
    }
}

```

Fig 5.196 Invalid quantity exception

5.9 File I/O / Serialization

OmniCommerce uses SQLite via JDBC for all data persistence. The JDBC connection follows a standard pattern using PreparedStatement to prevent SQL injection:

```

public static User login(String username, String password){
    try {
        Connection conn = DBConnection.getConnection();
        String sql = "SELECT * FROM users WHERE username = ? AND password = ?";
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setString(1, username);
        stmt.setString(2, hashPassword(password));

        ResultSet rs = stmt.executeQuery();
    }
}

```

Fig 5.197 PreparedStatement

Additionally, ImageUtil provides file I/O for product images. When an admin uploads a product image, ImageUtil copies the file to the data/product_images/ directory with a UUID filename and returns the path for storage in the database:

```

public static String copyProductImage(File source) throws IOException {
    if (source == null || !source.isFile()) return null;
    Files.createDirectories(PRODUCT_IMAGE_DIR);
    String extension = extensionOf(source.getName());
    String filename = UUID.randomUUID() + extension;
    Path target = PRODUCT_IMAGE_DIR.resolve(filename);
    Files.copy(source.toPath(), target, StandardCopyOption.REPLACE_EXISTING);
    return target.toString().replace(oldChar: '\\', newChar: '/');
}

```

Fig 5.198 ImageUtil

Chapter 6: Implementation & Key Code Highlights

6.1 Development Environment

Component	Details
IDE	Visual Studio Code with Java Extension Pack
JDK Version	Java 26.0.1 (build 26.0.1+8-34)
Database	SQLite via sqlite-jdbc-3.53.2.0.jar
GUI Framework	Java Swing (built into JDK no additional dependency)
Version Control	Git with GitHub (https://github.com/fahady724/eccommerce)
Operating System	Windows 11 (64-bit)
Build Tool	No build tool compiled directly with javac

6.2 Key Algorithm Implementations

SHA-256 Password Hashing

Passwords are hashed using Java's built-in MessageDigest class before being stored in the database. This ensures that even if the database file is accessed directly, user passwords cannot be read:

```
private static String hashPassword(String password) {
    try {
        java.security.MessageDigest md = java.security.MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(password.getBytes());
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) {
            sb.append(String.format(format: "%02x", b));
        }
        return sb.toString();
    } catch (Exception e) {
        return password;
    }
}
```

fig 6.1 SHA-256

Client-Side Search and Filter Algorithm

The StorefrontPanel implements real-time search with category filtering and multiple sort options entirely in memory, without additional database queries:

```
private void applyFilters() {
    if (productGrid == null) return;
    String query = searchField.getText().trim().toLowerCase(Locale.ROOT);
    String category = categoryBox.getSelectedItem().toString();

    List<Product> filtered = new ArrayList<>();
    for (Product product : allProducts) {
        String name = product.getName() == null ? "" : product.getName().toLowerCase(Locale.ROOT);
        String description = product.getDescription() == null ? "" : product.getDescription().toLowerCase(Locale.ROOT);
        boolean matchesText = query.isEmpty() || name.contains(query) || description.contains(query);
        boolean matchesCategory = "All Categories".equals(category) || category.equalsIgnoreCase(product.getCategory());
        if (matchesText && matchesCategory) filtered.add(product);
    }

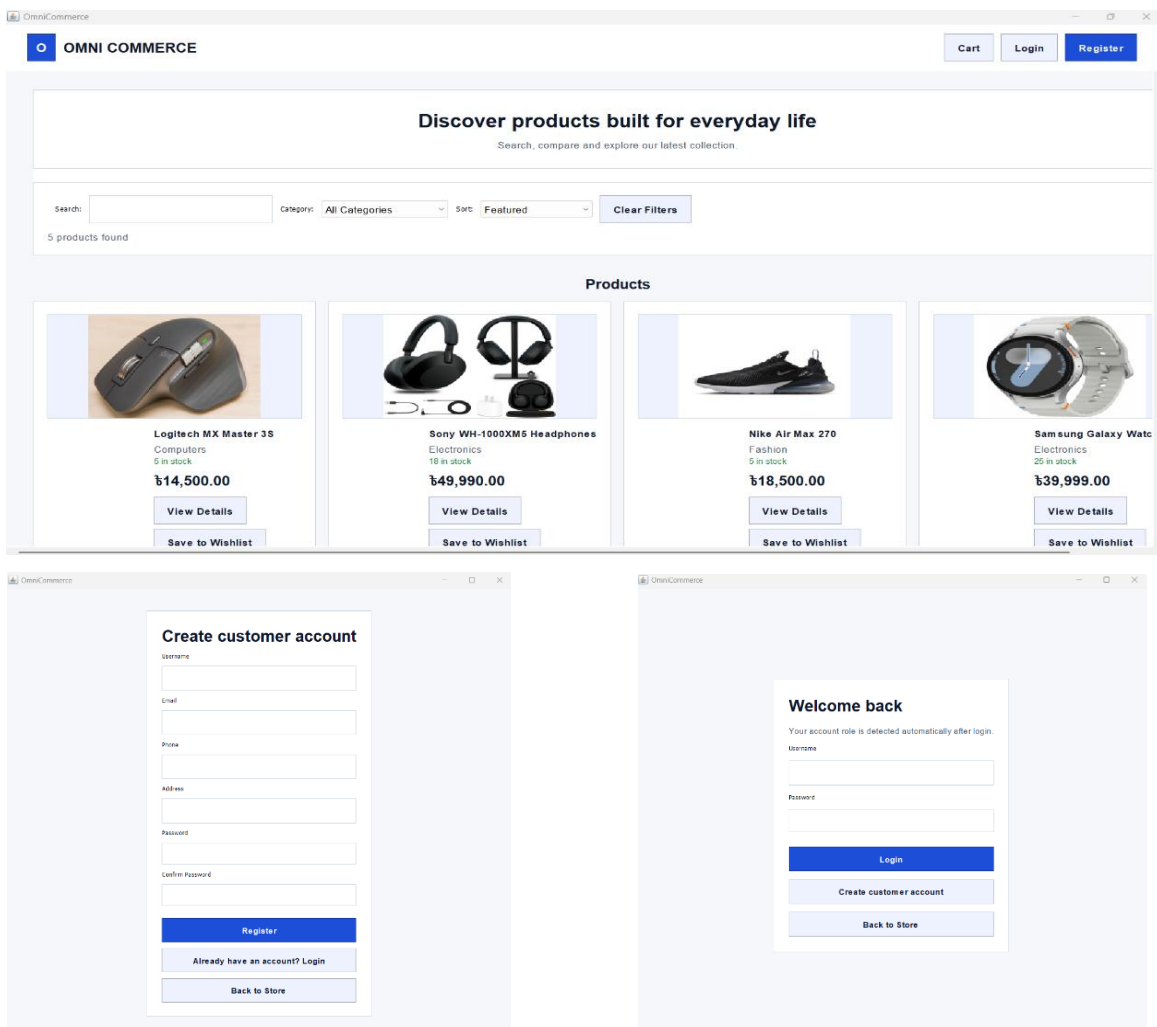
    String sort = sortBox.getSelectedItem() == null ? "Featured" : sortBox.getSelectedItem().toString();
    if ("Price: Low to High".equals(sort)) {
        filtered.sort(Comparator.comparingDouble(Product::getPrice));
    } else if ("Price: High to Low".equals(sort)) {
        filtered.sort(Comparator.comparingDouble(Product::getPrice).reversed());
    } else if ("Name: A to Z".equals(sort)) {
        filtered.sort(Comparator.comparing(Product::getName, String.CASE_INSENSITIVE_ORDER));
    }

    renderProducts(filtered);
}
```

fig 6.2 filtering

6.3 User Interface Design

The UI is built with Java Swing using a CardLayout navigation pattern. All screens share a single JFrame (AppFrame) and switch between panels using named cards. This prevents window proliferation and provides a smooth, application-like feel.



OmniCommerce - Shopping Cart

Shopping Cart

Product	Price	Quantity	Subtotal
Logitech MX Master 3S	₹14,500.00	2	₹29,000.00
Sony WH-1000XM5 Headphones	₹49,990.00	1	₹49,990.00
Nike Air Max 270	₹18,500.00	1	₹18,500.00

Total: ₹97,490.00

Remove Selected

Clear Cart

Checkout

OmniCommerce

Admin Dashboard

Back to Store

Logout

Products

Customers

Orders

Revenue

5

1

1

₹97,490.00

Product Management

ID	Name	Category	Price	Stock
5	Logitech MX Master 3S	Computers	₹14,500.00	3
4	Sony WH-1000XM5 Headphones	Electronics	₹49,990.00	17
3	Nike Air Max 270	Fashion	₹18,500.00	4
2	Samsung Galaxy Watch7	Electronics	₹39,999.00	25
1	iPhone 13	Mobile & Accessories	₹30,000.00	50

Refresh

Reports & Analytics

Manage Customers

Manage Orders

Add Product

Edit Selected

Delete Selected

OmniCommerce - Checkout

Checkout

Review your order and choose a payment method.

Delivery information

Customer

fahad12

Email

asfghuias@gmail.com

Phone

01972742013

Address

afdsaf

Product	Qty	Price	Subtotal
Logitech MX Ma...	2	₹14,500.00	₹29,000.00
Sony WH-1000X...	1	₹49,990.00	₹49,990.00
Nike Air Max 270	1	₹18,500.00	₹18,500.00

Payment method

Cash on Delivery

Mobile Banking

Credit Card

Pay when your order is delivered.

Back to Cart

Total: ₹97,490.00

Place Order

A central Theme class defines all visual constants:

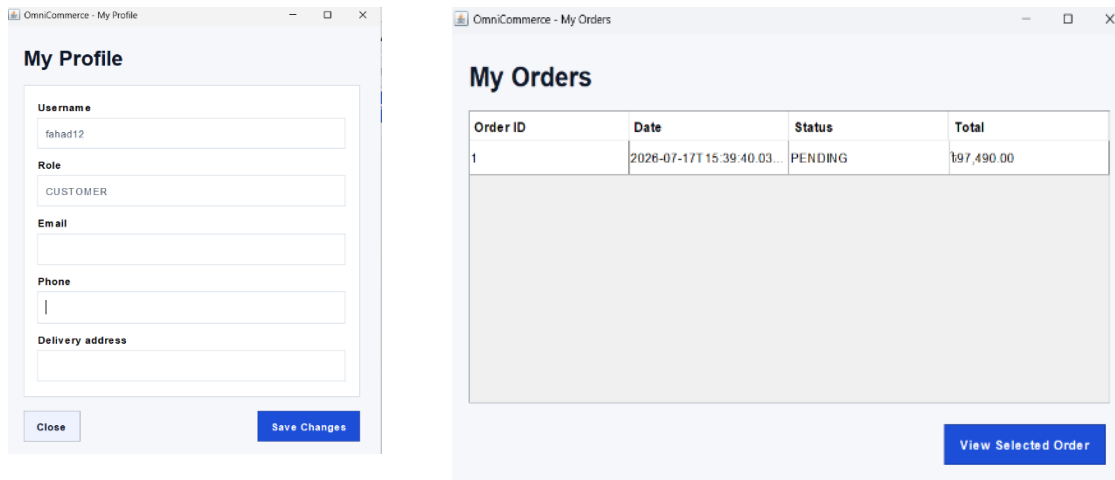
- Background: #F6F7FB (light grey)
- Surface: White with #CBD5E1 border
- Primary action: #1D4ED8 (blue)

- Danger action: #B91C1C (red)

- Success indicator: #15803D (green)

- Typography: SansSerif

- Title 28pt Bold, Heading
- 20pt Bold, Body
- 14pt Regular



Key screens include: Storefront (product grid with search/filter/sort), Login (centred card with Enter key support), Register (scrollable form), Admin Dashboard (summary stats + product table), Reports (tabbed analytics view with monthly sales, category performance, order status, low stock), Cart (table with total), and Checkout (split view showing delivery info and payment selection).

6.4 Data Persistence Implementation

All data persistence is handled through SQLite using the JDBC interface. The DBConnection class implements the Singleton pattern to ensure only one database connection is maintained throughout the application lifetime:

```
public class DBConnection {
    private static final String URL = "jdbc:sqlite:ecommerce.db";
    private static Connection connection = null;

    public static Connection getConnection() {
        if (connection == null) {
            try {
                connection = DriverManager.getConnection(URL);
                System.out.println("Database connected successfully!");
            } catch (SQLException e) {
                System.out.println("Connection failed: " + e.getMessage());
            }
        }
        return connection;
    }
}
```

fig 6.3 Database

DatabaseSetup uses CREATE TABLE IF NOT EXISTS for all tables, making it safe to call on every launch. It also handles schema migration by attempting to add new columns and silently ignoring the error if they already exist, allowing older database files to be upgraded automatically.

6.5 Authentication & Authorization

Authentication is handled by `AuthService.login()` which queries the database for a user with the given username and SHA-256 hashed password. If found, it reads the role field and returns either a Customer or Admin object accordingly.

Authorization is enforced at multiple levels:

- `AppFrame.isAdmin()` checks the current user's role before showing admin-only UI elements.
- `StorefrontPanel` hides "Add to Cart" and wishlist buttons for admin users.
- `CartFrame`, `CheckoutFrame`, and `OrdersFrame` redirect to login if no user is authenticated.
- `AdminDashboardPanel` is only accessible after a successful admin login attempting to navigate to it without admin credentials is not possible through the UI.

Chapter 7 Testing & Validation\

7.1 Testing Strategy

OmniCommerce was tested using manual functional testing, where each feature was exercised through the user interface to verify correct behaviour. Testing was conducted in three phases:

- Unit-level testing: Each service method was tested individually by calling it from Main.java with known inputs and verifying the database output using DB Browser for SQLite.
- Integration testing: Complete user flows were tested end-to-end (register → login → browse → add to cart → checkout → view order history).
- Edge case testing: Invalid inputs were deliberately entered to verify that validation and exception handling prevented crashes.

7.2 Test Class

Test ID	Description	Input	Expected Output	Actual Output	Pass/Fail
TC-01	Register with valid data	fahad12, pass12345, email, phone, address	Account created; redirect to login	Account created; redirect to login	PASS
TC-02	Register duplicate username	fahad12 (already exists)	Error: Username already exists	Error dialog shown	PASS
TC-03	Login with correct credentials	fahad12 / pass12345	Login success; storefront shown	Login success; storefront shown	PASS
TC-04	Login with wrong password	fahad12 / wrongpass	Error: Invalid username or password	Error dialog shown	PASS
TC-05	Login as admin	fahad / 14529882	Admin panel available in navigation	Admin panel shown	PASS
TC-06	Add product (admin)	Valid name, price, stock, category	Product appears in table and storefront	Product added successfully	PASS
TC-07	Add product with negative price	-500	Validation error: price cannot be negative	Error dialog shown	PASS

TC-08	Search products by name	"iPhone"	Only iPhone products displayed	Filtered results shown	PASS
TC-09	Add to cart (logged in)	iPhone 15, qty 1	Cart count increments; item in cart	Cart updated correctly	PASS
TC-10	Add to cart (not logged in)	Click Add to Cart	Prompt to login shown	Login prompt appeared	PASS
TC-11	Checkout with cash	Items in cart, select Cash	Order placed; cart cleared	Order created successfully	PASS
TC-12	Checkout with invalid card	15-digit card number	Error: Invalid card number	Error dialog shown	PASS
TC-13	Checkout with valid bKash	+8801972742013, bKash	Order placed (90% success rate)	Order placed	PASS
TC-14	Order more than stock	Request 100 items, only 5 in stock	Error: insufficient stock	InvalidQuantityException caught; error shown	PASS
TC-15	View order history	After placing order	Order appears in My Orders list	Order visible in history	PASS
TC-16	Update order status (admin)	Select order, set DELIVERED	Order status updated in database	Status updated and confirmed	PASS
TC-17	Delete product (admin)	Select existing product	Product removed from catalogue	Product deleted with confirmation	PASS
TC-18	Data persistence	Register, add to cart, close app, reopen	Cart contents and user account preserved	All data persisted	PASS
TC-19	Update profile	Change email and address	Profile updated in database	Profile saved successfully	PASS
TC-20	Wishlist add and move to cart	Save product, move to cart	Product removed from wishlist, added to cart	Wishlist and cart updated	PASS

7.3 Edge Case Testing

The following boundary and edge cases were specifically tested:

- Empty form submission: All forms display validation errors when required fields are left blank, and form submission is blocked.
- Concurrent cart updates: Adding the same product to the cart twice updates the quantity rather than creating a duplicate entry, due to the UPDATE logic in `CartService.addToCart()`.

- Zero stock product: Products with zero stock display "Out of stock" and the Add to Cart button is disabled.
- Mobile number normalisation: Both +8801972742013 and 01972742013 formats are accepted for bKash/Nagad payments.
- Database not found on first launch: DatabaseSetup.createTables() creates all tables automatically with IF NOT EXISTS, so the application works on a fresh machine with no database file.
- Wishlist duplicate: Attempting to add a product already in the wishlist shows an appropriate message rather than creating a duplicate, due to the UNIQUE constraint on (customer_id, product_id).

7.5 Bugs Found & Fixed

Bug ID	Description	Root Cause	Fix Applied
BUG-01	Admin login failed even with correct credentials	Admin.java returned "Admin" but login checked for "ADMIN"	Changed Admin role string to "ADMIN" in both constructors and getRole()
BUG-02	deleteProduct() silently failed without deleting	SQL was "DELETE products WHERE id=?" missing FROM keyword	Corrected to "DELETE FROM products WHERE id=?"
BUG-03	getTotalSales() returned rounded integer values	Result was stored in int variable instead of double	Changed result variable type to double; used rs.getDouble(1)
BUG-04	Duplicate products added on every app launch	seedProducts() in Main.java ran unconditionally	Added addProductIfMissing() helper that checks before inserting
BUG-05	Cart items lost when user logged out and back in	Cart was stored only in memory (Cart.java)	Replaced in-memory cart with CartService using SQLite cart table
BUG-06	Dashboard did not open after login	DashboardFrame was using DISPOSE_ON_CLOSE which closed the app	Changed to DISPOSE_ON_CLOSE on all sub-frames; kept main AppFrame with EXIT_ON_CLOSE
BUG-07	Order placed even when payment failed	CheckoutFrame submitted order before checking payment result	processPayment() called first; order only submitted if it returns true

Chapter 8 Results & Discussions

8.1 System Demonstration Summary

OmniCommerce is a fully functional, self-contained e-commerce order management system. The application launches with a public storefront displaying a product grid with search, category filter, and sort capabilities. Unregistered visitors can browse products and view details. Upon registration and login, customers gain access to cart management, wishlist, order placement with three payment methods, order history, and profile management.

The administrator account (username: fahad, created automatically on first launch) provides access to an admin dashboard with product CRUD operations including image upload, customer management, order status management, and a reports section showing monthly revenue, category performance, low stock alerts, and order status distribution.

All data persists between sessions through SQLite. The application runs entirely without internet access and requires only JDK 17+ and the included sqlite-jdbc JAR file.

8.2 Requirement Fulfill Metrix

Requirement ID	Feature	Status
FR-01	User registration	Met
FR-02	Role-based login	Met
FR-03	SHA-256 password hashing	Met
FR-04	Duplicate username detection	Met
FR-05	Profile update	Met
FR-06	Product CRUD with image	Met
FR-07	Product browse with search and filter	Met
FR-08	Sort products	Met
FR-09	Product details dialog	Met
FR-10	Add to cart with quantity	Met
FR-11	Cart management	Met
FR-12	Wishlist	Met

FR-13	Checkout with 3 payment methods	Met
FR-14	Payment validation	Met
FR-15	Order history	Met
FR-16	Order details	Met
FR-17	Admin order management	Met
FR-18	Customer management	Met
FR-19	Summary statistics	Met
FR-20	Reports and analytics	Met

8.3 OOP Concepts Coverage

OOP Concept	Implementation	Classes/Methods	Status
Encapsulation	Private fields, getters/setters, SHA-256 hashing	All model classes	Met
Inheritance	Two-level hierarchy for User and Payment	User→Customer/Admin; Payment→CashPayment/MobilePayment/CreditCardPay	Met
Abstraction (Abstract Class)	Abstract User and Payment classes with abstract methods	User.getRole(); Payment.processPayment()	Met
Abstraction (Interface)	Payable interface implemented by Payment	Payable → Payment → all payment types	Met
Polymorphism (Overriding)	processPayment() overridden in all three payment classes; getRole() overridden in Customer and Admin	CheckoutFrame uses parent reference; runtime dispatch	Met
Polymorphism (Overloading)	addProduct(), updateProduct(), searchProduct() each have 2 overloaded versions	ProductService	Met
Collections Framework	ArrayList for all product/order/cart lists; LinkedHashSet for categories	Throughout all service and UI classes	Met
Exception Handling	4 custom exceptions; try-catch throughout; transaction rollback on order failure	exception package; OrderService; CheckoutFrame	Met

File I/O / Database	SQLite persistence via JDBC; file I/O for product images	All service classes; ImageUtil	Met
Static Members	All service methods static; DBConnection.connection static; Theme constants static final	All service classes; DBConnection; Theme	Met

8.4 Limitations

- Payment simulation: All payments are simulated. No real payment gateway is integrated. The 90% success rate for card and mobile payments is random and does not reflect actual bank responses.
- Single-user session: The application supports only one logged-in user at a time. There is no multi-session or concurrent user support.
- No email notifications: Order confirmations, registration emails, and password reset functionality are not implemented.
- Static service methods: All service methods are static, which simplifies the code but makes unit testing with mock objects more difficult.
- No product reviews: Customers cannot leave product ratings or reviews.
- Image storage: Product images are stored as file paths in the database. If the application folder is moved, image paths may break.

Chapter 9 Conclusions & Future Work

9.1 Conclusion

OmniCommerce successfully demonstrates the application of all four Object-Oriented Programming pillars Encapsulation, Inheritance, Abstraction, and Polymorphism in a realistic, fully functional e-commerce order management system. The project goes beyond the minimum requirements by implementing a wish list, detailed analytics reporting, product image management, multiple payment methods with Bangladesh-specific mobile number validation, and a consistent, professional graphical user interface built with a central design system.

The use of SQLite ensures data persistence without requiring a database server installation, making the application easy to run on any machine with a JDK. The transaction-based order placement system protects data integrity by rolling back all changes if any step of the order process fails. SHA-256 password hashing ensures basic security for stored credentials.

Building OmniCommerce provided practical experience with Java Swing GUI development, JDBC database programming, layered software architecture, exception handling design, and the application of OOP principles to solve real business problems.

9.2 Individual Reflection

Building OmniCommerce was a significant learning experience across multiple dimensions. Starting with near-zero familiarity with Java GUI programming, I progressively built understanding through practice first learning how JFrame and components work, then how to connect Java to a database using JDBC, and finally how to structure a multi-layer application with clear separation of concerns.

The most challenging part was the checkout flow coordinating the payment validation, order insertion, order item insertion, stock reduction, and cart clearing as a single atomic operation with proper transaction rollback. Understanding why all of these steps needed to be wrapped in a single database transaction, and what would happen to data integrity if they were not, was a valuable lesson that goes beyond syntax and into real software engineering thinking.

I also learned that good OOP design decisions make code naturally easier to extend. When adding the Wishlist feature, I created a new WishlistService and WishlistFrame without

modifying any existing service classes a direct benefit of the single-responsibility principle. This experience showed me that OOP is not just a set of rules to follow for marks, but a practical approach to building software that can grow and change over time.

9.3 Future Enhancements

1. Real payment gateway integration: Integrate with the bKash Merchant API or a sandbox payment gateway to replace the simulated payment processing with actual transaction verification.
2. Email notification system: Implement order confirmation and registration emails using the JavaMail API, improving the customer experience with automated communication.
3. Product ratings and reviews: Allow customers to leave ratings (1-5 stars) and text reviews for products they have ordered, with the average rating displayed on product cards.
4. Discount and coupon system: Implement a coupon management system allowing admins to create discount codes with percentage or fixed-amount reductions and expiry dates.
5. Inventory alerts: Automatically notify the admin via an in-app notification when any product falls below a configurable low-stock threshold, rather than requiring them to check the reports page manually.
6. DAO layer refactoring: Introduce dedicated Data Access Object classes (UserDAO, ProductDAO, OrderDAO) to completely separate SQL from business logic, improving testability and maintainability.
7. Unit testing: Add JUnit test cases for all service classes, using an in-memory SQLite database to test database operations without affecting production data.

References

- [1] B. Eckel, Thinking in Java, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2006.

- [2] Oracle Corporation, "The Java Tutorials Learning the Java Language," Oracle, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>. [Accessed: July 2026].

- [3] Oracle Corporation, "Java SE 26 API Documentation," Oracle, 2026. [Online]. Available: <https://docs.oracle.com/en/java/javase/26/docs/api/>. [Accessed: July 2026].

- [4] SQLite Consortium, "SQLite Documentation," SQLite, 2024. [Online]. Available: <https://www.sqlite.org/docs.html>. [Accessed: July 2026].

- [5] xerial, "sqlite-jdbc: SQLite JDBC Driver," GitHub, 2024. [Online]. Available: <https://github.com/xerial/sqlite-jdbc>. [Accessed: July 2026].

- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.

- [7] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ: Prentice Hall, 2008.

Appendices

Appendix A Source Code

The complete source code is available on GitHub at:

<https://github.com/fahady724/eccommerce>

The repository contains:

- src/ All Java source files organised in model, service, ui, exception, util packages
- sqlite-jdbc-3.53.2.0.jar SQLite JDBC driver
- README.md Setup and run instructions
- .gitignore Excludes compiled files and database from version control

Appendix C Team Contribution

Member	Student ID	Tasks	Contribution
Fahad Fardin Younus	253174038	All design, implementation, testing, and 100% documentation	

Appendix D Development Log

Date	Activity	Outcome
June 2026	Project planning and technology selection	Chose Java Swing + SQLite; designed class hierarchy
June 2026	Model classes and exception classes implemented	User, Product, Order, Payment hierarchy complete
June 2026	Database setup and service layer implemented	AuthService, ProductService, OrderService, CartService complete
June 2026	WishlistService and ReportService implemented	Analytics and wishlist features complete
July 2026	GUI implementation storefront, login, register	Public-facing screens complete and functional
July 2026	GUI implementation admin dashboard, checkout	Admin features and checkout flow complete
July 2026	Bug fixing and testing	All 7 known bugs identified and fixed
July 2026	Report writing and GitHub cleanup	Report complete; repository organised